

EXPERIMENT NO.2

Aim: To Develop and Establish smart contract and chain code.

Theory:

Smart Contract:

A smart contract is a sort of program that encodes business logic and operates on a dedicated virtual machine embedded in a blockchain or another distributed ledger.

Step 1: Business teams collaborate with developers to define their criteria for the smart contract's desired behaviour in response to certain events or circumstances.

Step 2: Conditions such as payment authorization, shipment receipt, or a utility meter reading threshold are examples of simple events.

Step 3: More complex operations, such as determining the value of a derivative financial instrument, or automatically releasing an insurance payment, might be encoded using more sophisticated logic.

Step 4: The developers then use a smart contract writing platform to create and test the logic. After the application is written, it is sent to a separate team for security testing.

Step 5: An internal expert or a company that specializes in vetting smart contract security could be used.

Step 6: The contract is then deployed on an existing blockchain or other distributed ledger infrastructure once it has been authorized.

Step 7: The smart contract is configured to listen for event updates from an "oracle," which is effectively a cryptographically secure streaming data source, once it has been deployed.

Step 8: Once it obtains the necessary combination of events from one or more oracles, the smart contract executes.

Chaincode:

Chaincode is a program, written in Go, Node.js, or Java that implements a prescribed interface. Chaincode runs in a secured Docker container isolated from the endorsing peer process. Chaincode initializes and manages ledger state through transactions submitted by applications. A chaincode typically handles business logic agreed to by members of the network, so it may be considered as a “smart contract”. Ledger updates created by a chaincode are scoped exclusively to that chaincode and can’t be accessed directly by another chaincode. However, within the same network, given the appropriate permission a chaincode may invoke another chaincode to access its state.

Now, we will explore chaincode through the eyes of a blockchain network operator rather than an application developer. Chaincode operators can use this topic as a guide to how to use the Fabric chaincode lifecycle to deploy and manage chaincode on their network.

Deploying a chaincode

The Fabric chaincode lifecycle is a process that allows multiple organizations to agree on how a chaincode will be operated before it can be used on a channel. A network operator would use the Fabric lifecycle to perform the following tasks:

1. Install and define a chaincode
2. Upgrade a chaincode
3. Deployment Scenarios

Migrate to the new Fabric lifecycle

You can use the Fabric chaincode lifecycle by creating a new channel and setting the channel capabilities to V2_0. You will not be able to use the old lifecycle to install, instantiate, or update a chaincode on channels with V2_0 capabilities enabled. However, you can still invoke chaincode.

Install and define a chaincode

Fabric chaincode lifecycle requires that organizations agree to the parameters that define a chaincode, such as name, version, and the chaincode endorsement policy. Channel members come to agreement using the following four steps. Not every organization on a channel needs to complete each step.

- **Package the chaincode:** This step can be completed by one organization or by each organization.
- **Install the chaincode on your peers:** Every organization that will use the chaincode to endorse a transaction or query the ledger needs to complete this step.
- **Approve a chaincode definition for your organization:** Every organization that will use the chaincode needs to complete this step. The chaincode definition needs to be approved by a sufficient number of organizations to satisfy the channel's Life cycle Endorsement policy (a majority, by default) before the chaincode can be started on the channel.
- **Commit the chaincode definition to the channel:** The commit transaction needs to be submitted by one organization once the required number of organizations on the channel have approved. The submitter first collects endorsements from enough peers of the organizations that have approved, and then submits the transaction to commit the chaincode definition.

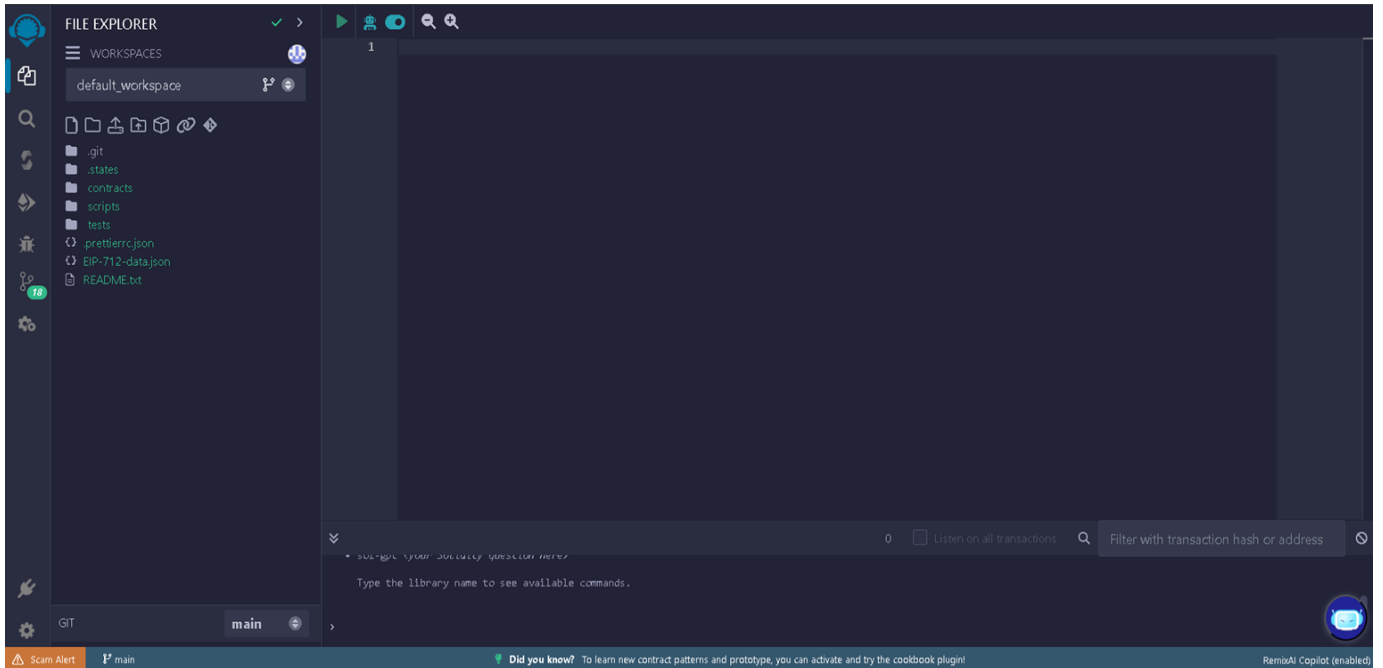
Implementation:

This playground provides us with all we need to write our first smart contract.

1. Let's start by deleting the file created for us by default. To do that, click on the first icon below the remix logo. Right-click on the file name in the explorer section and select delete:
2. Click OK in the pop-up menu.
3. Create a new file named Blockchain.sol by clicking the document icon marked red in the image below and type the name of the file in the space provided:
4. .sol is the extension used for solidity files. The blank space is where we will type our code. Solidity code always begins with the line below:

```
// SPDX-License-Identifier: UNLICENSED
```

Without this code, you will get an error. It is just like saying that you accept the terms and conditions of writing Solidity.



In the add BlockToChain, we start by incrementing the chainCount by one like this: `chainCount += 1;`
Next, add the block of a transaction to the chain like this:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract Blockchain {
    struct BlockStruct {
        uint256 index;
        uint256 timestamp;
        uint256 amount;
        address sender;
        address recipient;
    }
}
```

```
BlockStruct[] public chain;
uint256 public chainCount;
```

```
event BlockEvent(uint256 amount, address indexed sender, address indexed recipient);
```

```
constructor() {
    chainCount = 0; // Initialize chain count
}
```

```
function addBlockToChain(uint256 amount, address payable recipient) public {
    chainCount += 1;
    chain.push(
        BlockStruct(
```

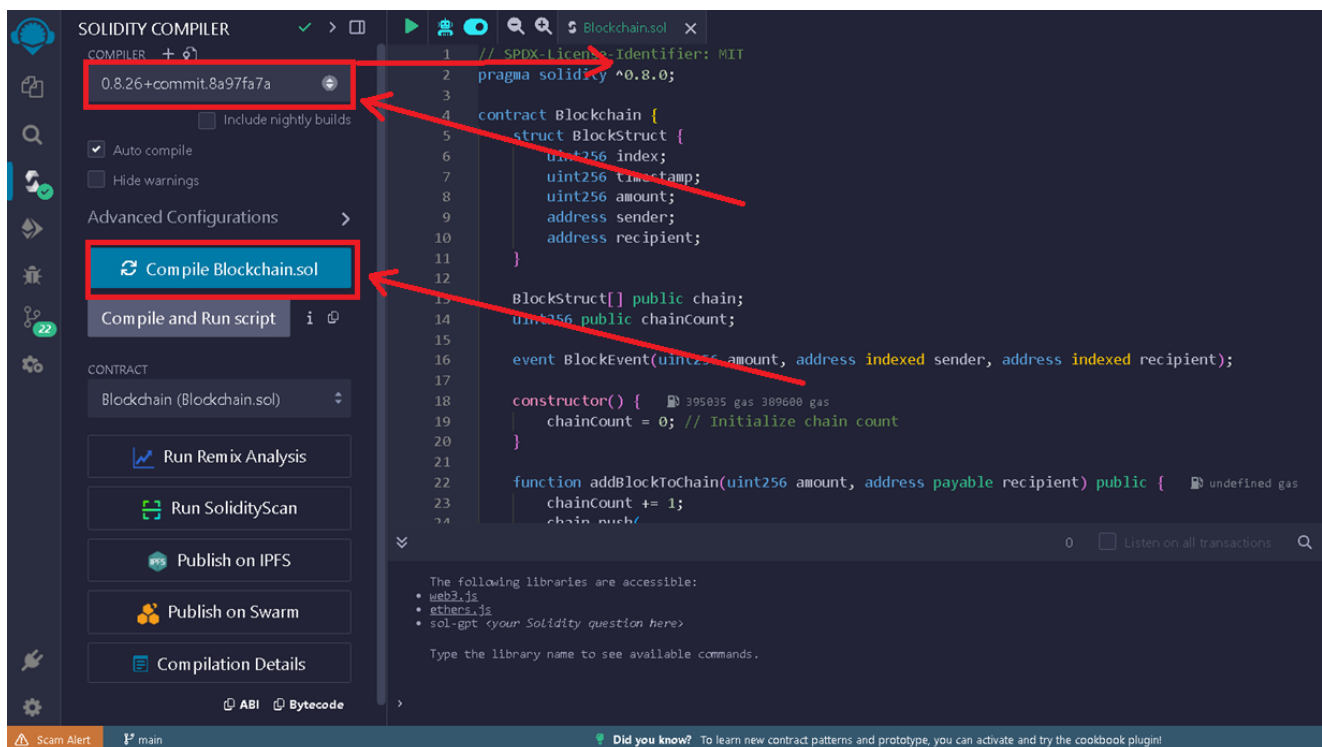
```

        chainCount,
        block.timestamp,
        amount,
        msg.sender,
        recipient
    )
);
emit BlockEvent(amount, msg.sender, recipient);
}

function getChain() public view returns (BlockStruct[] memory) {
    return chain;
}
}

```

Ensure that the solidity version selected matches the one you specified at the beginning of the smart contract. Then click the Compile Button



The compilation was successful since we have no errors.

Conclusion: Thus, we have successfully studied and developed smart contract and chain code.